

GSD DANCE STUDIO

Payment Receipt Generation System Using Python

Professional Project Documentation

Project Information	Details
Prepared By	P. Jayanth Abhinay
Organization / Institute	Hanominds learning technologies
Date	05-10-2026
Project Type	Beginner Python Mini Project
Tools Used	Python, Google Colab / Jupyter Notebook
Technology	Python Programming

1. Executive Summary

The GSD Dance Studio Payment Receipt System is a beginner-level Python project designed to generate a professional payment receipt for dance studio students.

The program collects basic student information such as student name, age, dance type, and session. It also collects payment-related information, including advance payment and payment mode.

A fixed monthly fee of ₹1,000 is used in the program. The system calculates the remaining balance using the advance payment entered by the user.

The program also generates a receipt number, displays student and payment details in a structured format, and determines the payment status as either "PAID" or "PARTIALLY PAID".

The project demonstrates beginner Python concepts including variables, data types, input(), type conversion, arithmetic operators, string methods, conditional statements, for loops, formatted output, and basic receipt generation.

The project is useful as a practical introduction to how Python can automate a simple real-world business process.

2. Introduction

2.1 What This Project Is About

This project creates a digital payment receipt for GSD Dance Studio using Python. Instead of manually preparing a receipt, the user enters student and payment information through the Python program.

2.2 Why This Project Is Important

Small businesses such as dance studios often need to maintain payment records. A simple Python program can help demonstrate how repetitive manual tasks can be automated.

2.3 Real-World Business Relevance

The system can be useful for recording student details, dance type, session, monthly fee, advance payment, remaining balance, payment mode, and payment status.

2.4 Expected Outcome

The expected outcome is a professional-looking console receipt containing the student's information and payment status.

3. Problem Statement

GSD Dance Studio needs a simple method for generating payment receipts. Manual receipt preparation can require repeatedly entering and calculating payment information.

The Python program solves this basic problem by accepting student information, accepting payment information, calculating the remaining balance, determining payment status, and generating a formatted receipt.

This project is a basic receipt automation system, not a complete database or accounting application.

4. Project Objectives

- To understand Python variables.
- To understand different data types.
- To accept user input.
- To perform arithmetic calculations.
- To use conditional statements.
- To use loops.
- To format output professionally.
- To calculate outstanding payment.
- To determine payment status.
- To generate a structured payment receipt.
- To demonstrate a real-world application of beginner Python concepts.

5. Dataset Description

Dataset Availability

Not Applicable. The uploaded notebook does not contain a CSV, Excel, SQL table, or other tabular dataset. Instead, the program receives information interactively through input().

Input Fields

Field	Type	Description
Student Name	Text/String	Name of the student
Age	Integer	Age of the student
Dance Type	Text/String	Type of dance selected
Session	Text/String	Morning or evening session
Monthly Fee	Integer	Fixed monthly fee of ₹1,000
Advance Payment	Float	Amount paid in advance
Payment Mode	Text/String	Cash or UPI
Receipt Number	String	Receipt identification number

Data Classification

Field	Classification
Student Name	Text
Age	Numerical
Dance Type	Categorical/Text
Session	Categorical/Text
Monthly Fee	Numerical
Advance Payment	Numerical
Payment Mode	Categorical/Text
Receipt Number	Text

Because the program does not store multiple records in a dataset, there are no dataset rows or columns to analyze.

6. Business Questions

1. What is the student's name?
2. What is the student's age?
3. What type of dance does the student attend?
4. Which session does the student attend?
5. What is the monthly fee?
6. How much advance payment has been made?
7. What is the remaining balance?
8. What payment mode was used?
9. Has the monthly payment been fully paid?
10. What is the receipt number?

Questions such as "Which dance type is most popular?" or "What is the monthly revenue?" cannot be answered from the current program because it does not store historical transaction data.

7. Tools and Technologies

Tool / Technology	Purpose	Real-World Use
Python	Main programming language	Business automation
input()	Collect user information	Data entry
Variables	Store information	Managing business data
Arithmetic Operators	Perform calculations	Payment calculation
Conditional Statements	Determine payment status	Business rules
for Loop	Repeat footer message	Repeated output
String Methods	Format receipt	Professional output
Jupyter / Google Colab	Run Python code	Development and testing

The uploaded project does not use Pandas, NumPy, Matplotlib, or Seaborn because it is not a data-analysis project.

8. Project Workflow

Start → Enter Student Details → Enter Payment Details → Set Monthly Fee → Calculate Remaining Balance → Generate Receipt Number → Determine Payment Status → Format Receipt → Display Receipt → End

Step 1 — Enter Student Details

The program collects student name, age, dance type, and session.

Step 2 — Enter Payment Details

The user enters advance payment and payment mode.

Step 3 — Set Monthly Fee

The program defines `monthly_fee = 1000`.

Step 4 — Calculate Remaining Balance

The program uses `remaining_balance = monthly_fee - advance_payment`.

Step 5 — Determine Payment Status

The program checks whether the remaining balance is zero.

Step 6 — Generate Receipt

The program displays all information using formatted console output.

9. Data Loading

There is no external dataset in the uploaded notebook. Therefore, functions such as `pd.read_csv()` and `pd.read_excel()` are not used.

User Input

```
student_name = input("Enter Student Name: ")
```

This allows the user to enter information directly.

Type Conversion

```
age = int(input("Enter Age: "))
```

```
advance_payment = float(input("Enter Advance Payment: ₹"))
```

This allows the program to perform mathematical calculations.

10. Data Exploration

Traditional data-analysis functions such as `head()`, `tail()`, `shape`, `columns`, `dtypes`, `info()`, and `describe()` are not applicable because the project does not contain a Pandas DataFrame.

The program instead works with variables such as `student_name`, `age`, `dance_type`, `session`, `monthly_fee`, `advance_payment`, and `payment_mode`.

11. Data Cleaning

Cleaning Activity	Status
Missing-value handling	Not implemented
Duplicate removal	Not applicable
Data-type correction	Partially handled using int() and float()
Empty-string validation	Not implemented
Extra-space cleaning	Not implemented
Inconsistent-value handling	Not implemented
Column-name correction	Not applicable
Unnecessary-column removal	Not applicable
Outlier detection	Not implemented

Important Limitation

The current program assumes that the user enters valid information. Entering text instead of a number for age, for example, could cause an error. A future version could add validation.

12. Exploratory Data Analysis

Traditional EDA is not applicable because no historical dataset exists. The current program performs individual transaction calculations instead.

Payment Calculation

```
remaining_balance = monthly_fee - advance_payment
```

Example: If Monthly Fee = ₹1,000 and Advance Payment = ₹500, the Remaining Balance = ₹500.

13. Bivariate Analysis

Bivariate analysis requires multiple records or observations to compare variables. The current notebook contains only a single interactive transaction at a time. Therefore, scatter plots, bar charts, box plots, and heatmaps are not included.

14. Multivariate Analysis

Multivariate analysis is not applicable to the current project because there is no historical dataset containing multiple observations.

Future versions could store Student Name, Age, Dance Type, Session, Monthly Fee, Advance Payment, Remaining Balance, Payment Mode, and Payment Status in a CSV or database.

15. Feature Engineering

The project does not perform traditional data-science feature engineering. However, it creates two calculated business fields.

Remaining Balance

```
remaining_balance = monthly_fee - advance_payment
```

This converts two payment values into a directly useful business value.

Payment Status

```
if remaining_balance == 0:
    status = "PAID"
else:
    status = "PARTIALLY PAID"
```

This converts the numerical balance into an easy-to-understand business status.

16. Data Visualization

No charts are included in the uploaded notebook. This is appropriate for the current application because the purpose is to generate an individual receipt rather than analyze historical data.

Possible future visualizations include monthly revenue, number of students by dance type, students by session, payment mode distribution, paid vs partially paid students, and outstanding payment totals. These require storing multiple transactions.

17. Business Insights

Insight 1 — Fixed Monthly Fee

The program uses a fixed monthly fee of ₹1,000.

Insight 2 — Payment Balance Can Be Calculated

The system calculates the outstanding amount using monthly fee minus advance payment.

Insight 3 — Payment Status Is Automated

The program identifies a fully paid transaction when the remaining balance equals zero. Otherwise, the status is PARTIALLY PAID.

Insight 4 — Receipt Information Is Structured

The receipt contains separate sections for student details, payment details, payment status, and signatures.

No broader business trends can be claimed because historical transaction data is not available.

18. Recommendations

Recommendation 1 — Store Transaction Records

Save every receipt to a CSV, Excel file, or database. Business benefit: the studio could maintain historical payment records.

Recommendation 2 — Add Input Validation

Validate age, advance payment, session, and payment mode. Business benefit: this would reduce incorrect data entry.

Recommendation 3 — Generate Unique Receipt Numbers

The current notebook uses GSD-001. A future version should generate a unique receipt number for every transaction.

Recommendation 4 — Add Date Automatically

A future version could automatically use the current date, reducing manual work and date-entry errors.

Recommendation 5 — Create a Payment Database

Store student and payment information in a structured database so the studio can analyze revenue, outstanding payments, registrations, and attendance.

19. Challenges Faced

- Manual Input — The user must enter each student's information manually.
- No Persistent Storage — The current program does not save transaction records after execution.
- Limited Validation — The program assumes valid user input.
- Fixed Monthly Fee — The monthly fee is directly defined as ₹1,000.
- Fixed Receipt Number — The receipt number is currently GSD-001 and does not automatically increment.

20. Conclusion

The GSD Dance Studio Payment Receipt System is a practical beginner Python project that demonstrates how programming can be used to automate a simple business task.

The program accepts student and payment information, calculates the remaining balance, determines payment status, and produces a structured payment receipt.

The project demonstrates variables, strings, integers, floating-point numbers, user input, type conversion, arithmetic operators, conditional statements, loops, string formatting, and console-based output.

The project is a receipt-generation application rather than a data-analysis project. It does not contain historical data, so statistical analysis, EDA, visualization, and business trend analysis cannot legitimately be performed.

21. Future Scope

- CSV/Excel Storage — Save every transaction into a CSV or Excel file.
- Database Integration — Use SQLite or another database to store student and payment records.
- Automated Receipt Numbers — Generate sequential or unique receipt IDs.
- Automatic Date and Time — Automatically record transaction date and time.

- **Payment Dashboard** — Create a Power BI dashboard showing total students, revenue, outstanding payments, paid transactions, dance-type distribution, session distribution, and payment-mode distribution.
- **Data Analysis** — Analyze monthly revenue, popular dance types, morning vs evening enrollment, payment collection trends, outstanding balances, and student age groups.
- **Machine Learning** — With sufficient historical data, explore student retention prediction, payment-delay prediction, enrollment forecasting, and revenue forecasting.

22. References

- **Uploaded GSD Dance Studio Python notebook** — primary project source.
- **Python Documentation** — recommended for extending the Python application.
- **Pandas Documentation** — recommended for future data storage and analysis.
- **NumPy Documentation** — recommended for future numerical analysis.
- **Matplotlib Documentation** — recommended for future visualization.
- **Seaborn Documentation** — recommended for future statistical visualization.

23. Project Summary

Category	Details
Project	GSD Dance Studio Payment Receipt
Programming Language	Python
Project Level	Beginner
Application Type	Receipt Generation
Monthly Fee	₹1,000
Input Method	User Input
Output	Formatted Payment Receipt
Payment Calculation	Monthly Fee – Advance Payment
Payment Status	PAID / PARTIALLY PAID
Data Storage	Not implemented
Dataset	Not available
EDA	Not applicable
Visualization	Not implemented
Database	Not implemented
Future Platform	Python + CSV/Excel/Database + Power BI

Final Project Statement

The GSD Dance Studio Payment Receipt System demonstrates how basic Python programming concepts can be applied to automate a real-world payment-receipt process. The project provides a foundation for developing a complete dance studio management, payment tracking, and analytics system.